

A journey from source systems to trusted, analysis-ready datasets

Data Ingestion & Transformation Pipelines

Outline

1	Define the Data Pipeline	Source systems → governed, analysis-ready datasets
2	Data pipeline design decisions	Batch/stream • full/incremental/CDC • push/pull
3	Design the Data Architecture	Bronze → Silver → Gold layers
4	Deliver Analysis-Ready Data	Clean, enrich, validate, and publish for analytics and ML
5	Data Processing using DataFrame	Transform, clean, enrich, validate, and publish data using DataFrame



Data pipeline design decisions

OLTP Captures Business Activity; OLAP Turns It into Insight

	OLTP — operational workloads	OLAP — analytical workloads
Purpose:	record day-to-day transactions.	find patterns and answer questions.
Typical action:	insert or update a few rows.	scan and summarize many rows.
Optimized for:	fast, reliable writes with low latency.	joins, trends, and aggregates.
Example:	place an order and update stock.	compare monthly sales by region.

Why separate them?

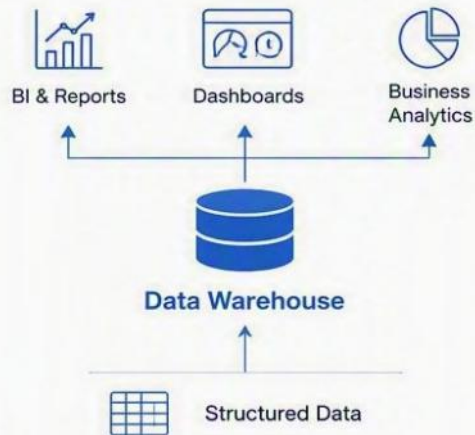
Prevent analytical workloads from slowing down operational transactions.

The business relies on both workloads: one supports daily operations, while the other enables analysis and insight

Data Warehouse vs Data Lake vs Data Lakehouse

DATA WAREHOUSE

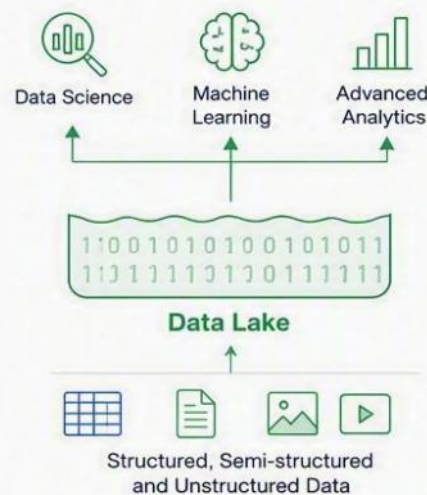
Structured data for reporting and business intelligence



	Data Type	Structured
	Schema	Schema-on-Write (Rigid)
	Performance	Fast for complex queries and aggregations
	Data Quality	High (governed, validated)
	Best For	Reporting, BI, dashboards, KPIs

DATA LAKE

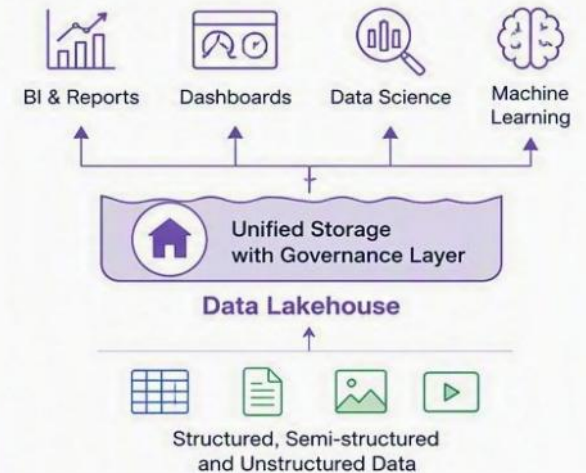
All raw data for flexibility and advanced analytics



	Data Type	Structured, semi-structured and unstructured
	Schema	Schema-on-Read (Flexible)
	Performance	Slower for complex queries (scan large volumes)
	Data Quality	Variable (raw data)
	Best For	Data science, ML, exploration, real-time data

DATA LAKEHOUSE

Best of both worlds: flexibility of a lake with the performance and governance of a warehouse



	Data Type	Structured, semi-structured and unstructured
	Schema	Schema-on-Read and Schema-on-Write
	Performance	Fast and scalable for BI, ML and analytics
	Data Quality	High (governed + flexible)
	Best For	BI, data science, ML, real-time analytics



Key Takeaway: Use a **Data Warehouse** for trusted reporting, a **Data Lake** for flexibility and exploration, and a **Data Lakehouse** when you need both performance and flexibility together.

The end-to-end data journey



✓ Evidence preserved

- Identify where data originates
- Raw records + source identifiers, arrival metadata
- Move new or changed data
- E.g. Checkout database, product catalog, inventory events

✓ Meaning clarified

- Clean, standardize, join, and apply business definitions
- E.g., Fix dates, join product data, calculate the line total
- Test rules and monitor quality
- E.g., Order ID present; quantity > 0

✓ Use governed

- Define data ownership and access controls
E.g., only the Finance team can access revenue data.
- Publish trusted data for use
- E.g., Curated orders feed a dashboard and stock alert

Five design aspects to settle before you ingest

Design aspect	Decide	Online orders example
1 Source ownership and support	Who owns it, who to call	Commerce team owns checkout data
2 Grain and primary key	What one row means, and its key	One row = one item in an order
3 Change behavior of records	Append only, or handle updates and deletes?	Not append only: an order can be updated or cancelled later
4 Freshness and schedule	How fresh must it be?	Sales report hourly is good enough
5 Data quality and recovery	What to do with bad rows	Set aside orders with no ID

Batch vs Streaming

Ask: How quickly do we need to act on this data?

Decision Factor	Batch	Streaming
Freshness needed	Hourly or daily updates	Seconds-to-minutes decisions
How it works	Process available data on a schedule	Process new events continuously
Practical example	Yesterday's sales report, ready by 7 a.m.	Block a suspicious card payment before it goes through
Cost and effort	Cheaper: if it fails, just run it again	Costlier: always on, harder to fix

Can the decision wait? Yes → Batch | No → Streaming

Use streaming when faster results lead to better decisions or actions.

Choose how much data to load

Ask: Do we reload everything, or only what changed?

Option	What it means	Use it when	Online-order example
Full load	Copy the whole table again	First load, or the table is small enough to copy quickly	Copy the whole product catalogue again each night
Incremental load	Copy only rows added or changed since last time	Too big to reload, and rows carry a reliable date or ID	Load only the orders placed since last night's run
	Stream every add, change, and delete	Data copies must stay in sync in near real time	Follow one order as it is placed, paid, or cancelled

Start simple: Full load → Incremental → CDC only when needed.

Always record what was loaded and when.

Push vs. pull: who starts the data movement?

Ask: Do we need the data immediately, or is a schedule enough?

Compare	Push	Pull
Who starts?	The source system	The data pipeline
When?	When something changes	On a schedule
Online-order example	The checkout app sends each order the moment it is placed	The pipeline asks the store for new orders once an hour
Best when	Fast updates are needed	Scheduled updates are enough
Watch for	Retries and duplicate events	Missed changes and load on the source

Quick rule: need it now → Push | a schedule is enough → Pull

Both need a plan for failures and duplicate data.



Medallion Architecture

Medallion Architecture



Bronze → Silver → Gold: data gets cleaner at each stage

Bronze

Keep it

Stored exactly as it
arrived

Silver

Fix it

Cleaned and de-
duplicated

Gold

Use it

Summarised for one
question

Online-order example

**“raw order files
dropped by each store”**

**one clean orders table,
one row per order line**

**daily sales by product,
ready for the
dashboard**

Raw → Clean → Useful

Never overwrite Bronze: it is the only way to rebuild Silver and Gold.

Bronze: the row plus tracking columns

Added to every row	Online-order example	Why it matters
source_name	orders_api	Says which system the row came from
ingested_at	2026-09-03 10:03	Shows when it arrived -> can help identify late data
load_id	batch_2026_09_03_01	Lets you replay just one bad run
change_type (new / changed / cancelled)	Order 5512 arrives as "new" at 10:03, then as "cancelled" at 11:40	Tells you the second row cancels the first, not a second sale

Bronze can be messy, but the original values are never changed.

Write Bronze once, never edit it. Fix data in Silver, so you can always rebuild.

What Silver actually changes

Transformation	What it means	Online-order example
Fix data types	Text becomes numbers, dates, booleans	"19.99" becomes 19.99; "2026-09-03" becomes a date
Handle missing values	Fill, flag, or quarantine blanks	Order with no customer_id goes to a quarantine table
Remove duplicates	Keep one row per business key	Same order sent twice by the API: keep the latest
Standardise values	One spelling, one unit, one code	"QA", "Qatar", "qatar" all become QAT
Apply business rules	Reject rows that cannot be true	quantity <= 0 or order_date in the future is rejected
Join reference data	Add trusted names and attributes	product_id 88 gains its name, category, and unit price

Every change is a rule you can explain and repeat.

Silver is where messy data becomes data people can trust.

Silver in practice: a Pandas example

17

Step	Pandas code	What it does
Load Bronze	<code>df = pd.read_csv("bronze/orders")</code>	Reads the raw rows as received
Fix numbers	<code>df["price"] = pd.to_numeric(df["price"])</code>	Bad prices become NaN, not a crash
Fix dates	<code>df["date"] = pd.to_datetime(df["date"])</code>	Text dates become real dates
De-duplicate	<code>df = df.drop_duplicates(["order_id", "line_id"])</code>	One row per order line
Validate	<code>df = df[df["quantity"] > 0]</code>	Removes impossible quantities
Join reference	<code>df = df.merge(products, on="product_id")</code>	Adds product name and category

Same four moves every time: load, fix, de-duplicate, validate.

Silver rules: what to do when data fails

18

Rule	Why it exists	What happens when it fails
order_id is not null	Without a key, the row cannot be traced	Row moves to quarantine for the source team
quantity > 0	A sale of zero or minus items is not real	Row is rejected and not counted in the daily report
price is a number	Text prices break every calculation	Value set to NaN, row flagged for review
order_date <= today	Future dates mean a broken clock or bad parse	Row is held back until the source is fixed
product_id exists in products	Unknown products break sales reporting	Row is kept but marked "unknown product"

Never delete a bad row silently - always record it.

A rejected row is information, not rubbish.

Gold answers a clear business question

1 ASK A QUESTION

“How much did each product sell every day?”

2 DEFINE ONE ROW

One product × one store × one date

3 PUBLISH THE ANSWER

DATE	STORE	PRODUCT	UNITS	REVENUE
5/9/2026	Doha 01	Laptop A	12	QAR 36,000

Used by: dashboard, data analyst, ML models

Gold is trusted when users agree on the question and each column's meaning.

Governance answers three simple questions

QUESTION	PLAIN MEANING	ONLINE-SHOP EXAMPLE
WHO can use it?	Who can view or change the data?	Analysts see sales; only finance sees payment details
WHAT is it?	What does it mean, and who owns it?	"daily_product_sales" owned by the sales data team
WHERE did it come from?	Which sources and steps created it?	Orders API → Silver orders → sales per store per day

Unity Catalog keeps access, meaning, and lineage in one place.

Good governance makes trusted data easier to use.

Data Types





Anatomy of Data

- **Data object:** An individual sample or entity
 - e.g., a customer, a transaction, or a sensor reading
 - Also known as a record, instance, sample, or entity
- **Features:** A specific property or characteristic of the data object
 - e.g., Age, Income, Temperature
 - Also known as attribute, variable, field, or characteristic
- **Raw vs. Derived Features:**
 - **Raw:** collected or measured value of an attribute according to an appropriate measurement scale
 - **Derived:** constructed from data in one or more raw features (e.g., calculating "Debt-to-Income Ratio" from "Total Debt" and "Annual Income")

Objects

Features

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Feature Types

Type	Subtype	Examples
Categorical (Qualitative)	Nominal	Product type, name
	Ordinal	Size measured as small < medium < large
	Binary	Spam email (yes/no, true/false, 0/1)
	Date / Time	Job start date
Numerical (Quantitative)	Discrete Countable values (integers)	Number of students in a class
	Continuous Measurable values (infinite decimals).	Height, weight, salary

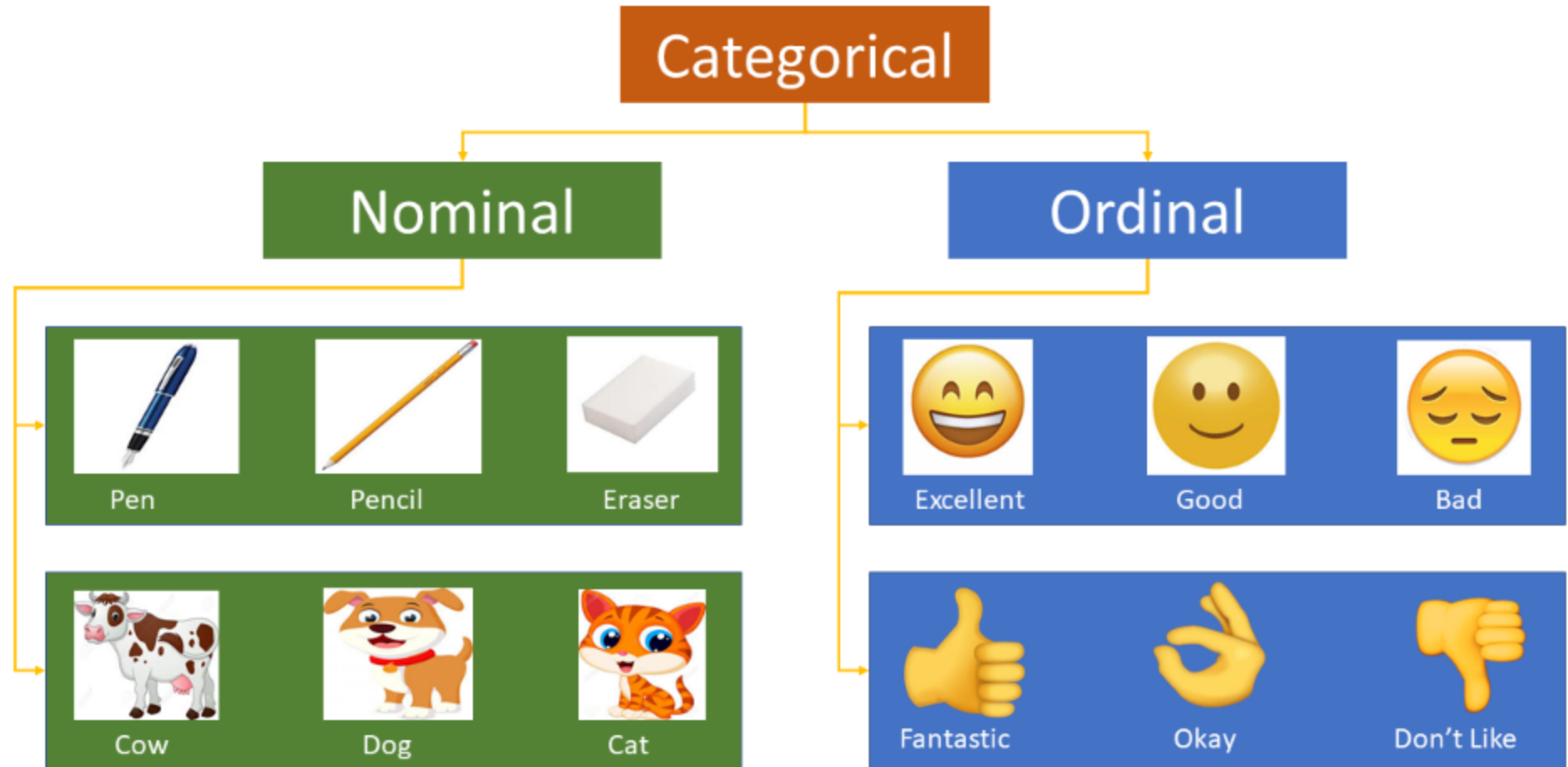
Understanding the type of variables is crucial for selecting appropriate statistical methods, visualization techniques, and ML algorithms

Categorical Features

- Categorical data are strings that represent qualitative data
 - Often selected from a group of categories, also called labels
- **Nominal**, e.g., country of birth, gender, eye color, etc.
 - No inherent order or ranking
 - Operators applicable: $=$, $\neq$
 - 1:1 transformation permissible, e.g. ID: 974 $\Rightarrow$ Qatar
- **Ordinal**, e.g. grade (A, B, C, D, F), degree (bachelor, master, PhD), height (tall, medium, short), etc.
 - Represent categories that can be meaningfully ordered
 - Operator applicable: $=$, $\neq$, $<$, $>$, $\geq$, $\leq$
 - Order-preserving transformation permitted,
 - e.g. height (tall, medium, short) to (1, 2, 3)



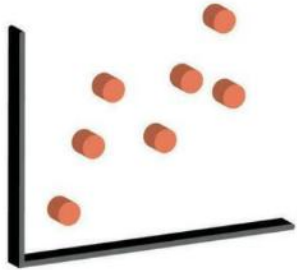
Nominal vs. Ordinal



Numerical Features

- **Discrete**

- Whole numbers (counts) typically integers
- E.g., The number of cars in a parking lot, the number of students in a class, or the count of items in a basket.



- **Continuous**

- Measurable numeric variable that may contain any value within a range
- Typically represented decimal numbers and fractions
- E.g., Height, weight, temperature, or distance



Dataset Types, Properties and Sources



Dataset Types (1 of 4)



Structured (Tabular) Data: Clearly defined rows and columns with a fixed schema

- E.g., Customer records in a SQL database, sales transactions in CSV files, student grades in a relational table



Semi-Structured Data: Flexible schema with self-describing structure (tags, keys)

- E.g., event logs stored as JSON files



Unstructured Text Data: Free-form text without a predefined schema

- E.g., Customer support tickets, emails, social media posts, open-ended survey responses

Dataset Types (2 of 4)



Image Data: Pixel-based visual data

- E.g., Medical X-rays, satellite imagery, product photos for visual inspection or classification



Audio Data: Sound signals typically represented as waveforms or spectrograms

- E.g., Call center recordings, voice commands for virtual assistants, meeting transcripts input



Video Data: Sequences of images with temporal context

- E.g., CCTV footage for security analytics, traffic monitoring videos, lecture recordings

Dataset Types (3 of 4)



Time-Series Data: Observations indexed by time, often with temporal dependencies

- E.g., Stock prices by minute, server CPU usage over time, IoT sensor readings collected every second



Transactional / Event Data: Discrete events generated by user or system actions

- E.g., E-commerce purchases, clickstream logs, login/logout events



Graph / Network Data: Entities and relationships represented as nodes and edges

- E.g., Social networks (users and friendships), citation networks, fraud detection graphs linking accounts and transactions

Dataset Types (4 of 4)

 **Geospatial Data:** Data associated with geographic locations and spatial relationships

- E.g., GPS traces from delivery vehicles, map coordinates of infrastructure, satellite-based land-use data.

 **Streaming / Real-Time Data:** Continuously generated data requiring near-real-time processing

- E.g., Live sensor feeds, financial market ticks, real-time user activity events.

 **Multimodal Data:** Combination of multiple data types in a single problem

- E.g., Product listings with text descriptions, images, and prices; medical records combining notes, images, and heart rate measurements time series

Dataset Properties

Size:

Measured in terms of the total number of records or total number of bytes, e.g. Small (MB), medium (GB) and large (TB)

Dimensionality:

Number of features

Density:

- How many of the data points in a dataset are filled with meaningful, non-zero values
- Sparse: The vast majority of values are zero/null
- Dense: Most data points are filled with meaningful, non-zero values.

Resolution:

- The level of detail at which data is recorded
- Related to the intended purpose

Data Density & Resolution

Property	Explanation	Real-World Example
Data Density	Describes how many of the data points in a dataset are filled with meaningful, non-zero values • Sparse: Vast majority of values are zero/null. Non-zero entries are rare but often highly informative • Dense: The opposite; most data points are filled with meaningful, non-zero values	• E-commerce: A matrix of all customers and all products. Most cells will be '0' because a customer only buys a tiny fraction of the total items available, making the data sparse • Demographics: A table of census respondents with columns for age, gender, and income level. Most cells would be filled, making the data dense
Resolution (or Granularity)	The level of detail at which data is recorded. The ideal resolution is dictated entirely by the analytical objective; too fine can be noisy, while too coarse can hide crucial patterns.	Sales Data: Transaction data can be aggregated at different resolutions. • Objective: Staffing for the lunch rush → High-resolution (per-minute sales) is needed • Objective: Analyzing annual growth → Low-resolution (monthly or quarterly sales) is sufficient and more efficient



Deliver Analysis-Ready Data

Six steps make data ready to use

ASK: What must change before a beginner can trust and use this data?

STEP	SIMPLE MEANING	ONLINE-ORDER EXAMPLE
CLEAN	Remove errors and repeats	Delete a duplicate order
STANDARDIZE	Use the same format and labels	Store every date as YYYY-MM-DD
INTEGRATE	Connect related data	Match each order to its customer
AGGREGATE	Choose the right level of detail	Total sales by day
ENRICH	Add a useful new field	Calculate delivery time
DOCUMENT	Explain meaning, quality, and owner	Define net_revenue and name its owner

Start with the user's question—not every dataset needs every step.

Use the variable type to choose the analysis

TYPE	ASK YOURSELF	ONLINE-SHOP EXAMPLE	USEFUL CHART
Nominal	Is it only a label?	payment method	bar chart
Ordinal	Can I put it in order?	low / medium / high rating	ordered bar
Discrete	Can I count whole items?	number of orders	bar or histogram
Continuous	Can I measure any value?	delivery time	histogram or box plot

Watch: `customer_id` is a label, even when stored as a number.

First identify the meaning; then choose the summary and chart.

Recognize common data structures

STRUCTURE	ONLINE-SHOP EXAMPLE	FIRST ENGINEERING TASK
Structured	orders table	check keys and updates
Semi-structured	JSON click event	handle nested fields and schema changes
Unstructured	review text or product image	add metadata and extract useful fields
Time-series	orders per minute	use event time and handle late data
Spatial	delivery GPS point	verify coordinates and precision
Event stream	clicks and purchases	keep order, remove duplicates, and replay

Beginner rule: identify the data's shape, then ask what can go wrong.

Follow one online order through the pipeline

BEGINNER QUESTION: What happens to order 1042 from source to report?

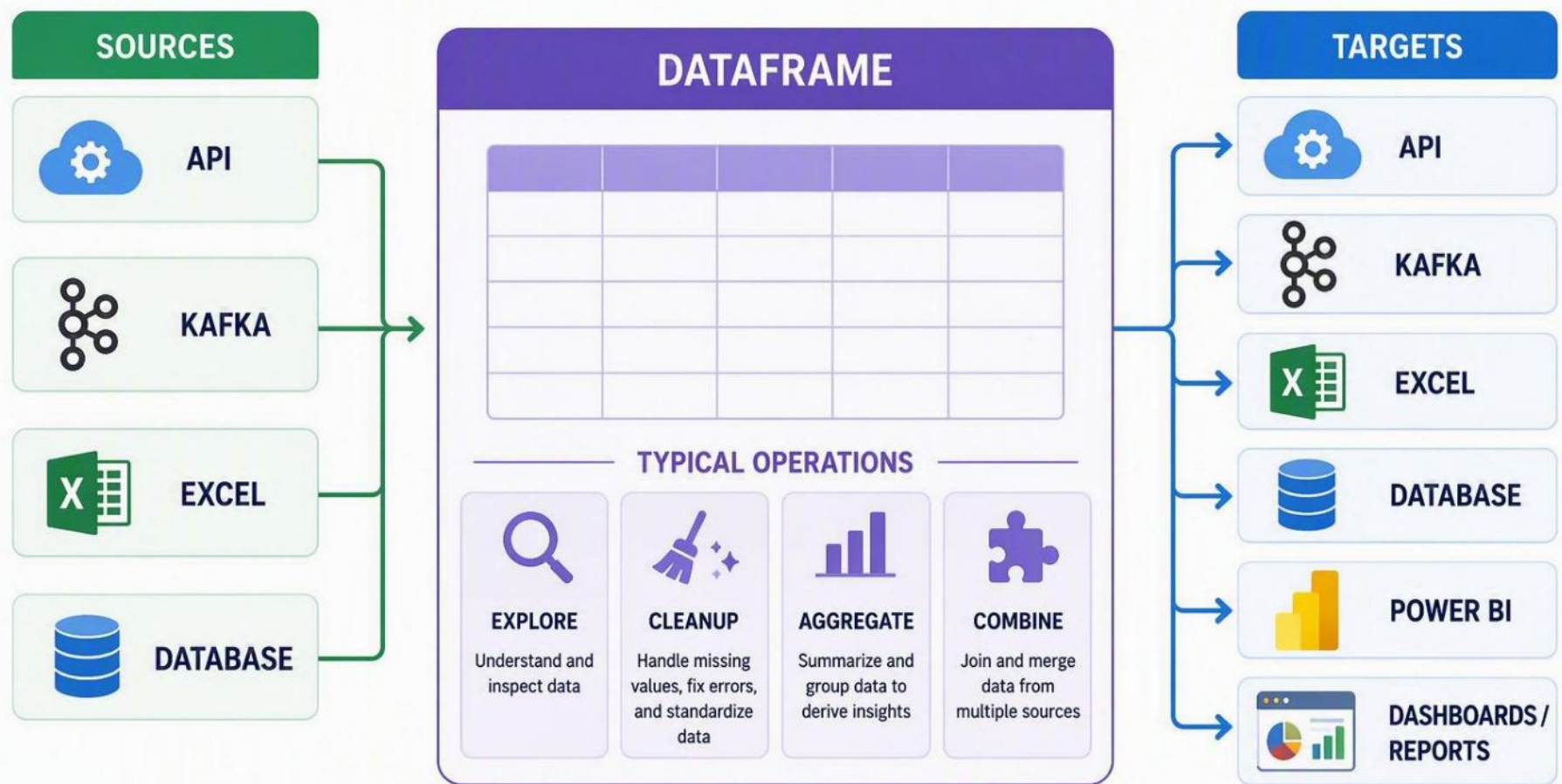
STAGE	WHAT HAPPENS	EXAMPLE OUTPUT
SOURCE	The store records a purchase	order 1042 arrives
BRONZE	Keep the raw event and tracking details	raw order + arrival time + source
SILVER	Fix types, remove repeats, match customer	one clean valid order
GOLD	Summarize for a business question	daily sales by product

If processing fails, replay from Bronze instead of asking the source to resend everything.



Data Processing using DataFrame

DataFrame = Core of ETL and Analytics



ONE STRUCTURE. MANY PURPOSES. DataFrame connects diverse sources to multiple targets through consistent operations—powering both data engineering and analytics.

A DataFrame brings a table into Python memory

Think of it as a Python table: labeled rows, named columns, and mixed data types.

FROM FILE TO DATAFRAME

`orders.csv` → `read_csv` → `DataFrame`

Index = row labels • Columns = fields

index	order_id	channel	amount	status
0	1041	Web	120	Complete
1	1042	Store	85	Pending
2	1043	App	60	Complete

LOAD, INSPECT, SELECT, FILTER

```
import pandas as pd

orders = pd.read_csv(
    "orders.csv")

orders.head()
orders.info()

complete = orders.loc[
    orders["status"]
    == "Complete",
    ["order_id",
     "channel", "amount"]
]
```

Pandas runs eagerly on one machine: each step executes now, using that machine's memory.

Process columns as a pipeline - not row by row

A clear chain shows the order: **filter** → **clean** → **derive** → **group** → **summarize** → **sort**.

ONE PANDAS PIPELINE

```
summary = (orders
    .loc[orders["status"] == "Complete"]
    .dropna(subset=["amount"])
    .drop_duplicates("order_id")
    .assign(net_amount=lambda d:
        d["amount"] * 0.95)
    .groupby("channel", as_index=False)
    .agg(orders=("order_id", "nunique"),
        revenue=("net_amount", "sum"))
    .sort_values("revenue", ascending=False)
)
```

READ THE CHAIN

- 1 Keep complete orders
- 2 Handle missing + duplicates
- 3 Create net_amount
Group by channel
- 5 Count orders + sum revenue
- 6 Put highest revenue first

JOIN ANOTHER TABLE

```
orders.merge(customers,
    on="customer_id", how="left")
```

Learn the DataFrame pattern once: Pandas, PySpark, and Polars reuse the same core idea.